from \ to	D_1	D_2	D_3	D_4
D_1	3	1		
D_2	5	1		
D_3	7	1		
D_4	8	1		
	$\leq C$	$\leq C$	$\leq C$	$\leq C$

Goal: Each round of token sending,

Each column needs to stay under

device capacity $= C$

Initially: capacity $= [C, C, C, C]$

Algorithm:

1. For each column, get # of >0.
   Call this number = # of senders_with_tokens_left

   ☆ Q: What if capacity = 0?
   A: To address this, add a max(..., 1) AND

2. Compute max(capacity // (# senders_with_tokens_left), 1)
   = budget

   (D, D) (D,)

3. Compute minimum(matrix, budget)
   = tokens_to_send

   Add a cap here

   3.1: cumsum(tokens_to_send, axis=0) ⇒ cap by capacity ⇒ diff
   Idea: Try to send D0 tokens first when capacity = 0

4. Compute : tokens_to_send.sum(axis="from")
   to get the number of tokens sent to each device

5. capacity = capacity − 4

6. Compute : matrix = matrix − tokens_to_send

7. If ($\boxed{\text{capacity}}$ still has nonzeros) AND

(matrix still has nonzeros),

go back to the step 1

Algorithm Part 2:

After we obtain $(\overset{\#\text{ of devices}}{\overset{\shortparallel}{D}}, \underset{\text{from}}{D}, \underset{\text{to}}{D})$ tokens_to_send matrix, we can use this to obtain

$(\underset{\text{from}}{D}, \underset{\text{to}}{D}, \#\text{ local experts})$ matrix

Idea: We always send tokens from the lower expert index first (greedily).

Implementation plan:

1. cumsum$(D, D, \#\text{ local experts}), \text{axis}=2)$

2. Apply cap by (D, D) tokens-to_send

to $(D, D, \#\text{ local experts})$

3. diff ((D, D, # local experts) , axis=2)

Output : 3